

TurnosRed — Informe técnico de la Actividad 2

Refactor de errores, recurso Médico, validación con Zod, filtros y colección de Postman · Integraciones Web

Repositorio: github.com/Luismcbo/turnos-red · **Stack:** Node.js 22 + TypeScript + Express + Zod + Socket.IO · **Autor:** Luis Valenzuela · **Fecha:** 23/09/2026

1. Resumen

La Actividad 2 amplía y refactoriza el backend de la Actividad 1 **sobre el mismo repositorio**: no se creó un proyecto nuevo. Se centralizó el manejo de errores con un formato único, se agregó el recurso **Médico** vinculable a los turnos, se incorporó validación de entrada con **Zod** (incluida la especialidad con grafía exacta), filtros por query params en la capa de servicios y una **colección de Postman** exportable con tests automatizados. Todo se verificó ejecutando la colección completa con Newman: **31 requests y 136 assertions, 0 fallas**.

Requisito de la consigna	Implementación	Dónde
1. Refactor de estructura y errores	Arquitectura en capas en inglés (+ <i>schemas</i>), verbos/códigos 200-201-204-400-404-500, middleware de errores al final de la cadena con formato único	src/errors, src/middlewares
2. Recurso Médico	CRUD completo en /medicos y <i>medicold</i> en turnos	src/*/medico.*
3. Validación con Zod	Schemas de Turno y Médico; especialidad exacta; middleware que convierte <i>ZodError</i> al error estándar	src/schemas, validate.ts
4. Filtros por query params	GET /turnos y GET /medicos filtran en la capa de servicios (sin endpoints nuevos)	src/services
5. Colección de Postman	31 requests, variables dinámicas, <i>pm.test</i> , ejemplos guardados	postman/
6. Documentación	README actualizado + sección "Uso de Inteligencia Artificial"	README.md
7. Commits por etapa	6 commits que continúan el historial de la Actividad 1	git log

2. Historial de commits (Actividad 2)

Commit	Mensaje
81d9864	refactor: arquitectura en inglés, errores centralizados y DELETE 204
d511f79	feat: validación con Zod, schemas y middleware de ZodError
20f8327	feat: recurso Médico (CRUD completo) y vínculo turno-médico
f8e1f72	feat: filtros por query params en turnos y médicos (capa de servicios)
142b66e	feat: colección de Postman con tests, variables dinámicas y Saved Responses
1ea7285	docs: actualizar README (Actividad 2) y agregar sección de uso de IA

Nota: Zod se implementó antes que Médico (orden distinto al de la lista de la consigna) para que el recurso Médico naciera ya validado, sin escribir validación manual descartable.

3. Arquitectura en capas y manejo de errores

Los nombres de las capas están en inglés y se agregó **src/schemas** como contrato de validación de entrada. Los términos de dominio (Turno, Médico, paciente...) se mantienen porque forman parte del contrato de la API (/turnos, /medicos).

routes	validate() (Zod)	controllers	services	models
--------	------------------	-------------	----------	--------

Flujo de una petición: *routes* → *middleware validate (schemas)* → *controller* → *service* → *model*. Los controladores no tienen try/catch: los servicios y la validación lanzan errores que recoge la cadena de errores.

Cadena de errores (al final de src/app.ts)

```
app.use(notFoundHandler); // ruta inexistente -> NOT_FOUND
app.use(zodErrorHandler); // ZodError -> VALIDATION_ERROR con details por campo
app.use(errorHandler); // formato final SIEMPRE { status, message, code, details }
```

Toda falla de la API sale con { "status", "message", "code", "details": [] }. Los errores desconocidos responden 500 *INTERNAL_ERROR* sin filtrar detalles internos (solo se registran en el log).

Código	Status	Cuándo
VALIDATION_ERROR	400	Body, query o params inválidos (Zod) o referencia inválida (medicoid inexistente)
INVALID_JSON	400	El cuerpo de la petición no es un JSON válido
NOT_FOUND	404	El recurso o la ruta no existen
INTERNAL_ERROR	500	Error no controlado

Verbos HTTP y códigos de estado

Método	Turnos (/turnos)	Médicos (/medicos)	Éxito	Errores
GET (lista)	/turnos	/medicos	200	400 (filtro inválido)
GET (uno)	/turnos/:id	/medicos/:id	200	400 (id inválido), 404
POST	/turnos	/medicos	201	400
PUT	/turnos/:id	/medicos/:id	200	400, 404
DELETE	/turnos/:id	/medicos/:id	204 (sin cuerpo)	400, 404

4. Validación con Zod

Los schemas de src/schemas se aplican con el middleware `validate({ params, query, body })` **antes** del controlador, que recibe datos ya validados y transformados (fecha ISO, hora HH:mm, confirmado booleano). Reglas destacadas:

- **documento**: string de formato libre (1–30 caracteres): admite 31654210, 31.654.210 o DNI 31654210; un número JSON se rechaza.
- **especialidad**: enum exacto *Clínica médica, Pediatría, Odontología, Nutrición*. Variantes como *PEDIATRÍA* o *pediatria* se rechazan.
- Objetos *strict*: un campo desconocido se rechaza ("Campo no permitido"). Fechas inexistentes (31/02) y horas fuera de rango también.
- Mensajes de Zod en español (z.locales.es) y *details* con { location, field, message }.

Evidencia 1 — especialidad “PEDIATRÍA” (caso pedido en la consigna)

```
POST /turnos
{"paciente": "Ana Lopez", "documento": "39887766", "especialidad": "PEDIATRÍA", "fecha": "20/08/2026", "hora": "11.00", "confirmado": true}

<- 400 Bad Request
{
  "status": 400,
  "message": "La peticion contiene datos invalidos (1 error)",
  "code": "VALIDATION_ERROR",
  "details": [
    {
      "location": "body",
      "field": "especialidad",
      "message": "Debe ser exactamente uno de: \"Clínica médica\", \"Pediatría\", \"Odontología\", \"Nutrición\"
(respetando mayúsculas y tildes)"
    }
  ]
}
```

Evidencia 2 — varios campos inválidos en una sola petición

Body enviado: paciente="", documento=123, especialidad="pediatria", fecha="31/02/2026", hora="25:00", confirmado="quizas", extra=true → 400, message: "La peticion contiene datos invalidos (7 errores)". Un detalle por campo:

location	field	message
body	paciente	No puede estar vacío
body	documento	Entrada inválida: se esperaba texto, recibido número
body	especialidad	Debe ser exactamente uno de: "Clínica médica", "Pediatría", "Odontología", "Nutrición" (respetando mayúscul...
body	fecha	Fecha inválida. Use DD/MM/YYYY o YYYY-MM-DD y una fecha real del calendario
body	hora	Hora inválida. Use HH:mm o HH.mm (00:00 a 23:59)
body	confirmado	Debe ser booleano (true/false) o "sí"/"no"
body	extra	Campo no permitido

5. Recurso Médico y vínculo con turnos

CRUD completo con la misma arquitectura en capas. Campos: *nombre*, *apellido*, *matricula*, *especialidad* (enum exacto), *disponible* (por defecto *true*) y *email* opcional. Los turnos aceptan *medicoId*: se valida que el médico exista (si no, 400 con el campo *medicoid* en *details*) y PUT con *medicoId*: null lo desvincula. Al **eliminar un médico**, sus turnos se conservan sin médico y se emite *turno:actualizado* por el bus de eventos (así los servicios no se importan entre sí y no hay dependencia circular). Ejemplo de error de validación en Médico:

location	field	message
body	nombre	No puede estar vacío
body	apellido	Entrada inválida: se esperaba texto, recibido indefinido
body	matricula	Entrada inválida: se esperaba texto, recibido indefinido
body	especialidad	Debe ser exactamente uno de: "Clínica médica", "Pediatría", "Odontología", "Nutrición" (respetando mayúscul...
body	email	Inválido dirección de correo electrónico

POST /medicos { nombre: "", especialidad: "Cardiología", email: "no-es-un-mail" } → 400

6. Filtros por query params (capa de servicios)

No se crearon endpoints nuevos: `turno.service.listTurnos(filtros)` y `medico.service.listMedicos(filtros)` aplican los filtros presentes (combinados con AND). En filtros la especialidad tolera mayúsculas y tildes (*Pediatria*), mientras que en el body es exacta. Fecha acepta DD/MM/YYYY o YYYY-MM-DD.

```
GET /turnos?especialidad=Pediatria&fecha=14/08/2026&medicoId=1
<- 200 OK
[
  {
    "id": 102,
    "paciente": "Carlos Ruiz",
    "documento": "31654210",
    "especialidad": "Pediatria",
    "fecha": "2026-08-14",
    "hora": "10:00",
    "confirmado": true,
    "medicoId": 1
  }
]
```

```
GET /medicos?especialidad=Odontologia&disponible=true
<- 200 OK
[
  {
    "id": 2,
    "nombre": "Martín",
    "apellido": "Suárez",
    "matricula": "MP-1002",
    "especialidad": "Odontología",
    "disponible": true,
    "email": "martin.suarez@turnosred.test"
  }
]
```

Filtros inválidos (?especialidad=Cardiologia&fecha=32/13/2026&medicoId=abc) → **400**, un detalle por parámetro con *location*: *query*:

location	field	message
query	especialidad	Especialidad desconocida. Valores admitidos (sin distinguir mayúsculas ni tildes): Clínica médica, Pediatrí...
query	fecha	Fecha inválida. Use DD/MM/YYYY o YYYY-MM-DD y una fecha real del calendario
query	medicoId	Debe ser un entero positivo

7. Colección de Postman

Archivos: `postman/turnos-red.postman_collection.json` y `postman/turnos-red.postman_environment.json` (*baseUrl*, *medicold*, *turnold*). Los ids se encadenan con `pm.environment.set` (y también como variable de colección, para que funcione sin environment activo). Cada request tiene tests de status code, *Content-Type*, esquema JSON (`pm.response.to.have.jsonSchema`) y reglas de negocio, más un **ejemplo guardado** con la respuesta real del servidor (listo para Mock Server).

Carpeta (orden de ejecución)	Requests	Assertions OK	Fallas
1. Médicos	11	51/51	0
2. Turnos	16	70/70	0
3. Limpieza y errores generales	4	15/15	0
Total	31	136/136	0

Resultado real de `npm run test:postman` (Newman) sobre un servidor recién iniciado; también corre en verde sin environment seleccionado.

Detalle por request (Newman)

Request	HTTP	OK
POST /medicos - Crear médico (201)	201	5/5
GET /medicos - Listar médicos (200)	200	4/4
GET /medicos?especialidad&disponible - Filtrar médicos (200)	200	4/4
GET /medicos/:id - Obtener médico (200)	200	4/4
PUT /medicos/:id - Actualizar médico (200)	200	4/4
POST /medicos - Especialidad "PEDIATRÍA" mal escrita (400)	400	6/6
POST /medicos - Campos inválidos y faltantes (400)	400	5/5
GET /medicos?disponible=quizas - Filtro inválido (400)	400	5/5
GET /medicos/abc - Id no numérico (400)	400	5/5
GET /medicos/:id - Médico inexistente (404)	404	5/5
PUT /medicos/:id - Médico inexistente (404)	404	4/4
POST /turnos - Crear turno con médico (201)	201	6/6
GET /turnos - Listar turnos (200)	200	4/4
GET /turnos?especialidad&fecha&medicold - Filtrar turnos (200)	200	4/4
GET /turnos/:id - Obtener turno (200)	200	4/4
PUT /turnos/:id - Actualizar turno (200)	200	4/4

Request	HTTP	OK
POST /turnos - Especialidad "PEDIATRÍA" mal escrita (400)	400	6/6
POST /turnos - Múltiples campos inválidos (400)	400	5/5
POST /turnos - medicold inexistente (400)	400	5/5
POST /turnos - JSON mal formado (400)	400	4/4
GET /turnos - Filtros inválidos (400)	400	5/5
PUT /turnos/:id - Body vacío (400)	400	5/5
GET /turnos/:id - Turno inexistente (404)	404	4/4
PUT /turnos/:id - Turno inexistente (404)	404	4/4
DELETE /turnos/:id - Eliminar turno (204)	204	2/2
GET /turnos/:id - Turno ya eliminado (404)	404	4/4
DELETE /turnos/:id - Eliminar dos veces (404)	404	4/4
DELETE /medicos/:id - Eliminar médico (204)	204	2/2
GET /medicos/:id - Médico ya eliminado (404)	404	4/4
DELETE /medicos/:id - Eliminar dos veces (404)	404	4/4
GET /ruta-inexistente - Ruta no encontrada (404)	404	5/5

8. Uso de Inteligencia Artificial

Herramienta principal: **Claude Code** (Claude Sonnet 5) trabajando sobre el repositorio. Las filas son ejemplos completados con lo que la herramienta generó en esta sesión; la columna *Ajuste manual aplicado* queda para completar por quien entrega el trabajo (el detalle completo está en el README).

Tarea	Herramienta	Prompt (resumen)	Respuesta generada	Ajuste manual
Refactor de errores: formato único y middleware centralizado	Claude Code (Sonnet 5)	"Middleware de errores centralizado que devuelva SIEMPRE { status, message, code, details }; códigos consistentes; 204 en DELETE."	AppError + ErrorCode, errorHandler/notFoundHandler al final de la cadena, servicios que lanzan NOT_FOUND, controladores sin try/catch, DELETE 204. Verificado con curl.	(a completar)
Validación con Zod (especialidad exacta, errores por campo)	Claude Code (Sonnet 5)	"Schemas de Zod para Turno y Médico; especialidad exacta (rechazar PEDIATRÍA); middleware que transforme los ZodError al formato estándar con details por campo."	Schemas strict, validate(), zodErrorHandler. Al probar se corrigió el mensaje de <i>confirmado</i> (z.union no aplicaba el texto: se pasó a z.custom) y se detectó que curl en Windows enviaba tildes en ANSI (pruebas con fetch de Node).	(a completar)
Colección de Postman con tests y ejemplos guardados	Claude Code (Sonnet 5)	"Colección exportable con variables, pm.test de status y esquema JSON, happy path y errores, y Saved Responses para Mock Server."	Script generador que ejecutó cada request contra el servidor real para guardar respuestas auténticas; 31 requests / 136 assertions; corrida Newman con 0 fallas.	(a completar)

9. Decisiones de diseño y conclusión

- **PUT** actualiza los campos enviados y rechaza un body vacío; **DELETE** responde 204 y repetirlo da 404.
- **medicoid** **inexistente** en un turno → 400 (dato inválido del cliente), no 404.
- **Especialidad** exacta en el body, tolerante en filtros y en el archivo turnos.json de las sedes (se traduce a su grafía canónica).
- El estado vive **en memoria** (los JSON son datos semilla); se reinicia al reiniciar el servidor.

Conclusión: la API responde de forma consistente ante cualquier falla, valida estrictamente lo que recibe, permite vincular turnos con médicos y filtrar listados, y queda respaldada por una colección de Postman reproducible que corre en verde de punta a punta.